

系统开发工具基础实验报告

第 1 周

姜又萌 25040031018

2026 年 8 月 24 日

1. 实验目的

本次实验是《系统开发工具基础》第 1 周的内容，围绕开发者日常必备的三类工具链展开：Shell 命令行、Git 版本控制、LaTeX 文档排版。实验旨在：

1. 掌握类 Unix Shell (bash) 的基本用法，能够导航文件系统、处理文件名中的空格与隐藏文件，并用管道组合多个命令完成文本处理；
2. 理解 Git 的分支与合并机制，亲手制造并解决一次合并冲突，理解冲突产生的根源；
3. 学会用 LaTeX 编写文档，掌握公式、表格、交叉引用三大核心功能，并能从命令行编译出 PDF；
4. 从”能运行”走向”可复现、可协作、可持续”的工程习惯，为后续每周的工具链学习打下基础。

2. 实验环境

- 操作系统：Windows 10 (64 位)
- Shell 环境：Git Bash (bash / MSYS2，满足课程要求的类 Unix Shell)
- 版本控制：Git 2.52 (命令行)
- LaTeX 发行版：TeX Live 2026 (编译器 XeLaTeX；自动化工具 latexmk)
- 编辑器：TeXworks / VS Code
- 编程语言：Shell (bash)、LaTeX

3. 课上练习

本部分为第 1 周课上完成的 4 道题，已交助教检查并讲解。

3.1.q01: 含空格文件名的批量整理

3.1.1.题目

在当前目录新建 q01，并完全使用命令行完成下列任务：

- 创建 input/docs 和 input/tmp；创建 input/docs/notes one.txt（内容为 alpha 和 beta 两行）、input/docs/.secret.txt（内容为 hidden）、input/tmp/empty.txt（空文件）以及 input/run.log（任意两行日志）；
- 显示 q01 的绝对路径，并以长格式列出 input 下全部项目，包含隐藏文件；
- 把 input 下所有.txt 文件复制到 work/学号/，保留相对目录结构并正确处理名称中的空格；
- 将复制后的目录权限设为 750、普通文件权限设为 640；生成 inventory.txt，列出相对路径和字节数。

3.1.2.解题思路

本题考察四点：**空格文件名的处理**（用引号包裹，防止 bash 按空白拆词）、**隐藏文件**（文件名以点开头，需 ls -a 显示）、**保留目录结构的复制**（用 cp -parents）、以及**权限与清单生成**（chmod 数字法与 find -printf）。

3.1.3.核心代码

```
1 mkdir -p input/docs input/tmp
2 echo -e "alpha\nbeta" > "input/docs/notes_one.txt"
3 echo hidden > input/docs/.secret.txt
4 touch input/tmp/empty.txt
5 printf 'info:\nstart\ninfo:\ndone\n' > input/run.log
6 pwd
7 ls -la input
8 find input -name "*.txt"
9 find input -name "*.txt" -exec cp --parents {} work/25040031018/ \;
10 find work -type d -exec chmod 750 {} \;
11 find work -type f -exec chmod 640 {} \;
12 find work -type f -printf "%p%s\n" > inventory.txt
13 ls -lR work
14 cat inventory.txt
```

3.1.4.运行结果

```
JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课堂/q01
$ ls -la input
total 1
drwxr-xr-x 1 JYM 197121 0 Aug 24 15:05 ./
drwxr-xr-x 1 JYM 197121 0 Aug 24 15:22 ../
drwxr-xr-x 1 JYM 197121 0 Aug 24 15:03 docs/
-rw-r--r-- 1 JYM 197121 23 Aug 24 15:05 run.log
drwxr-xr-x 1 JYM 197121 0 Aug 24 15:04 tmp/

JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课堂/q01
$ ls -la input/docs
total 2
drwxr-xr-x 1 JYM 197121 0 Aug 24 15:03 ./
drwxr-xr-x 1 JYM 197121 0 Aug 24 15:05 ../
-rw-r--r-- 1 JYM 197121 7 Aug 24 15:03 .secret.txt
-rw-r--r-- 1 JYM 197121 11 Aug 24 15:01 'notes one.txt'

JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课堂/q01
$ cat inventory.txt
work/25040031018/input/docs/.secret.txt 7
work/25040031018/input/docs/notes one.txt 11
work/25040031018/input/tmp/empty.txt 0
```

图 1: q01 运行结果截图

3.1.5.总结与收获

本次实验掌握了 Shell 处理文件系统的基础操作，记录几个关键问题与解答：

Q1: `echo -e` 和 `printf` 都能产生换行，有什么区别？**A:** 在认识 `\n` 这件事上两者等价；但 `printf` 支持格式化占位符（如`%.2f`）且天生认识转义，`echo -e` 依赖 `-e` 且可移植性稍差。涉及格式化输出必须用 `printf`。

Q2: 为什么用 `find -printf` 生成清单，而不是手动 `printf` 写进文件？**A:** `find -printf` 输出的是文件系统的实时真实数据（路径、字节数），会自动匹配隐藏文件（如`.secret.txt`）；手动 `printf` 会漏掉隐藏文件，且字节数容易写错。

Q3: `chmod 750` 和 `640` 怎么读？**A:** 权限数字 = 读 (4)+ 写 (2)+ 执行 (1) 相加。`750` = 属主 `rwX`、组 `r-X`、其他人无；`640` = 属主 `rw-`、组 `r-`、其他人无。

3.2.q02: 随机访问日志统计

3.2.1.题目

在 q02 中创建 `access.csv`，使用给定 8 行日志数据（含表头），编写 `analyze.sh` 完成统计：

- 脚本接收一个 CSV 文件路径；文件不存在时将错误写入 `stderr` 并返回非零退出码；
- 使用管道以及 `awk`、`sort`、`head` 等工具，输出 HTTP 5xx 数量最多的前 2 个 path；按次数降序，次数相同时按 path 字典序；
- 输出全部数据行的平均 `latency_ms`，保留两位小数；表头不得计入；
- 分别对 `access.csv` 和一个不存在的文件运行脚本；不得使用 Python、电子表格或手工统计。

3.2.2. 解题思路

本题核心是管道思想：把”过滤、排序、计数、再排序、取头”串成一条命令。用 awk 按逗号切列（-F,），以 NR>1 跳过表头、\$4 ~ /^5/ 筛出 5xx 状态码；再用 sort + uniq -c 计数、sort 多键排序满足”次数降序、同次数按字典序”；head -2 取前两个。平均延迟用 awk 累加 + END 块 + printf "%.2f" 保留两位小数。脚本还需检查参数个数（\$#）与文件是否存在，错误走 stderr 并返回非零退出码。

3.2.3. 核心代码

```
1  #!/bin/bash
2  set -euo pipefail
3  if [ $# -ne 1 ]; then
4      echo "用法: _$0_<CSV 文件路径>" >&2
5      exit 1
6  fi
7  CSV="$1"
8  if [ ! -f "$CSV" ]; then
9      echo "错误: 文件 _$CSV_ 不存在" >&2
10     exit 1
11 fi
12 awk -F, 'NR>1&&_4_~_^5/_{print _3_}' "$CSV" \
13 | sort | uniq -c \
14 | sort -k1,1nr -k2,2 \
15 | head -2 \
16 | awk '{print _2_,_1_}'
17 awk -F, 'NR>1{sum+=_5_;_count++;}END{printf _"%.2f\n",_sum/_count_}' "$CSV"
```

3.2.4. 运行结果



```
JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课堂/q02
$ bash analyze.sh access.csv
/api/orders 3
/api/users 2
193.75

JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课堂/q02
$ bash analyze.sh no_such.csv
错误: 文件 no_such.csv 不存在

JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课堂/q02
$ echo $?
1
```

图 2: q02 运行结果截图

3.2.5. 总结与收获

本题核心是 Shell 的管道思想，把过滤、排序、计数串成一条命令。记录几个关键问题与解答：

Q1: `cat` 不是用来输出文件内容的吗？为什么 `cat > access.csv << EOF` 是用来写入的？

A: `cat` 本质是”输入到输出”的搬运工。`cat >` 文件把标准输入重定向进文件（> 覆盖），所以用来写入；`cat` 文件把文件输出到屏幕。区别在于箭头方向，命令本身不变。

Q2: 把脚本里 `exit 1` 那段直接贴进终端，为什么窗口直接退出了？**A:** `exit 1` 在脚本里是结束脚本并返回状态码 1；但直接贴进交互终端，`exit` 就是退出整个 shell，所以窗口关了。正确做法是写进.sh 文件用 `bash` 运行。

Q3: `uniq` 之前为什么要先 `sort`？**A:** `uniq` 只合并相邻的重复行，若相同行被隔开就数不准。所以去重计数前必须先 `sort` 让相同行聚在一起，这是 `sort | uniq -c` 的死规矩。

Q4: 题目”平均延迟”是在哪里实现的？**A:** 在脚本最后一行用 `awk` 统计平均延迟，命令如下：

```
awk -F, 'NR>1{s+=$5;c++;}END{printf "%.2f",s/c}' "$CSV"
```

`NR>1` 跳过表头，累加第 5 列（latency），`END` 用 `%.2f` 保留两位小数输出均值。

3.3.q03: 制造、解决并解释一次合并冲突

3.3.1.题目

在 q03 中从零创建一个 Git 仓库，并主动制造、解决一次合并冲突：

- 初始化仓库，在 main 创建 `config.txt`（内容 `mode=normal`），完成初始提交；
- 创建 feature-a 把内容改为 `mode=safe` 并提交；从初始 main 创建 feature-b，把内容改为 `mode=fast` 并提交；
- 回到 main，先合并 feature-a，再合并 feature-b；解决冲突时保留 `mode=safe`，并另加一行 `note=reviewed`；
- 确认工作区干净，运行 `git log -all -graph -decorate -oneline` 查看历史。

3.3.2.解题思路

冲突的根源在于两条分支都修改了同一文件的同一行。feature-a 和 feature-b 都从初始 main 分叉，分别把 `config.txt` 的 `mode` 改成 `safe` 和 `fast`。合并 feature-a 时 main 一步未动，是快进（Fast-forward）合并，无冲突；而合并 feature-b 时 main 已前进（含 feature-a 改动），与 feature-b 对同一行动过手，git 无法自动决定，于是产生冲突。冲突时 git 用 `<<<<<<<`、`=====`、`>>>>>>>` 把两分支版本都写进文件，需要人工裁决。

3.3.3.核心代码

```
1 git init -b main
2 printf 'mode=normal\n' > config.txt
3 git add config.txt
4 git commit -m "init: config mode=normal"
5 git switch -c feature-a
6 printf 'mode=safe\n' > config.txt
7 git add config.txt
8 git commit -m "feature-a: mode=safe"
9 git switch main
```

```

10 git switch -c feature-b
11 printf 'mode=fast\n' > config.txt
12 git add config.txt
13 git commit -m "feature-b: mode=fast"
14 git switch main
15 git merge feature-a
16 git merge feature-b
17 printf 'mode=safe\nnote=reviewed\n' > config.txt
18 git add config.txt
19 git commit -m "merge feature-b: keep safe, add note"
20 git status
21 git log --all --graph --decorate --oneline

```

3.3.4. 运行结果



```

JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课堂/q03 (main)
$ git status
On branch main
nothing to commit, working tree clean

JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课堂/q03 (main)
$ git log --all --graph --decorate --oneline
*   ebec049 (HEAD -> main) merge feature-b: keep safe, add note
| \
|  * 074b74e (feature-b) feature-b: mode=fast
|  * | 40719c3 (feature-a) feature-a: mode=safe
| /
* d38a287 init: config mode=normal

```

图 3: q03 运行结果截图

3.3.5. 总结与收获

本题主动制造并解决了一次 Git 合并冲突。记录几个关键问题与解答：

Q1: 为什么第一次合并没冲突，第二次就出问题了？ A: 合并 feature-a 时 main 一步未动，属于快进（Fast-forward）合并，直接前移指针即可，无冲突；而合并 feature-b 时 main 已前进（含 feature-a 改动），与 feature-b 都改了 config.txt 同一行，git 无法自动裁决，于是冲突。冲突本质是两个分支改了同一文件的同一行。

Q2: 冲突怎么解决？解释要怎么说？ A: 人工编辑文件：保留想要的版本（mode=safe），删掉 <<<<<<<、=====、>>>>>>> 三个标记，再加内容，然后 git add（确认解决）+ git commit。解释要说清“为什么冲突（改同一行）、冲突时发生了什么（标记写入文件）、怎么解决（人工裁决 + add + commit）”。

Q3: config.txt 是用来干嘛的？ A: 它只是个练习道具文件。真实开发中配置文件恰好是冲突的重灾区，用它模拟冲突很贴切。

3.4. q04: 修复并构建一页技术说明

3.4.1. 题目

将给定 LaTeX 模板保存为 q04/report.tex，补全内容并从命令行构建 PDF：

- 加入一个带编号和 label 的公式，并在正文中使用 ref 或 eqref 引用；
- 加入一个 2 行 2 列表格，设置 caption 和 label，并在正文中引用该表；
- 使用 latexmk -pdf -halt-on-error 或 tectonic 生成 report.pdf；
- 确认 PDF 能打开，且交叉引用不显示??。

3.4.2. 解题思路

LaTeX 的交叉引用机制：先用 `\label{名字}` 给公式/表格起名，正文用 `\eqref{名字}`（公式）或 `\ref{名字}`（表格）引用。公式用 `equation` 环境自动编号，表格用 `table` 环境（管 caption 与 label）+ `tabular` 环境画格子。第一遍编译时引用尚未解析（显示??），第二遍才替换为真实编号，因此要编译两遍。为支持中文姓名，需使用 XeLaTeX 引擎并加载 `ctex` 宏包。

3.4.3. 核心代码

```

1 \documentclass{article}
2 \usepackage{amsmath}
3 \usepackage{ctex}
4 \begin{document}
5 \title{Tool Report}
6 \author{姓名-学号}
7 \maketitle
8 \section{Result}
9
10 Formula \eqref{eq:mode} shows the rule for choosing the mode.
11 \begin{equation}
12     \text{mode} =
13     \begin{cases}
14         \text{safe} & \& \text{if config is safe} \\
15         \text{fast} & \& \text{otherwise}
16     \end{cases}
17     \label{eq:mode}
18 \end{equation}
19
20 Table \ref{tab:config} lists the fields of the modified config file.
21 \begin{table}[h]
22     \centering
23     \caption{Config fields after merge}
24     \label{tab:config}
25     \begin{tabular}{|c|c|}
26         \hline
27         mode & safe \\
28         \hline
29         note & reviewed \\
30         \hline
31     \end{tabular}
32 \end{table}
33 \end{document}

```

编译命令：

```
1 xelatex -halt-on-error report.tex
2 xelatex -halt-on-error report.tex
```

3.4.4. 运行结果

Tool Report

姓名—学号

2026 年 8 月 31 日

1 Result

Formula (1) shows the rule for choosing the mode.

$$\text{mode} = \begin{cases} \text{safe} & \text{if config is safe} \\ \text{fast} & \text{otherwise} \end{cases} \quad (1)$$

Table 1 lists the fields of the modified config file.

表 1: Config fields after merge

mode	safe
note	reviewed

图 4: q04 编译结果截图

3.4.5. 总结与收获

本题用 LaTeX 构建了含公式、表格、交叉引用的技术说明页。记录几个关键问题与解答：

Q1：交叉引用为什么不会显示??? **A：**第一遍编译时引用尚未解析，位置先写?? 并把编号信息记进.aux 文件；第二遍编译读回.aux，才把?? 替换成真实编号。所以需要**编译两遍**，两遍后引用正常、无??。

Q2：\eqref 和\ref 有什么区别? **A：**\eqref 专用于公式，输出带括号的编号（如 (1)）；\ref 通用（表格、图、章节），输出不带括号的编号（如 1）。公式用\eqref 更符合数学书写习惯。

Q3：为什么中文要用 XeLaTeX + ctex? **A：**默认的 pdflatex 引擎不支持中文（遇到中文姓名会报错）；XeLaTeX + ctex 宏包天然支持中文，能正常编译含中文姓名的文档。

4.课后练习

本部分为课后完成的 12 个练习实例，覆盖 Shell（1-7）、Git（8-10）、LaTeX（11-12）三块，每题均编写代码并实际运行验证。以下为各实例题目及运行截图。

4.1.实例 1：通配符批量操作

题目：在 hw1 中创建 1.txt、2.txt、3.txt、a.md 四个文件，用 Shell 通配符与 for 循环将所有 .txt 文件批量重命名为.log（a.md 保持不变）。

```
JYM@DESKTOP-OPGLD3I MINGW64 ~  
$ cd D:/@作业/系统开发基础/课后实例/hw1  
  
JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课后实例/hw1  
$ touch 1.txt 2.txt 3.txt a.md  
  
JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课后实例/hw1  
$ for f in *.txt;do  
> mv "$f" "${f%.txt}.log"  
> done  
  
JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课后实例/hw1  
$ ls  
1.log 2.log 3.log a.md
```

图 5: 实例 1 运行截图

4.2.实例 2：通配符展开验真

题目：在 hw2 中创建 src 和 dst，在 src 放 3 个.txt，用通配符复制到 dst，再用花括号展开 a,b,c，区分通配符（匹配文件）与花括号（列出清单）。

```
JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课后实例/hw2  
$ mkdir src dst  
  
JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课后实例/hw2  
$ touch src/a.txt src/b.txt src/c.txt  
  
JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课后实例/hw2  
$ cp src/*.txt dst/  
  
JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课后实例/hw2  
$ ls dst  
a.txt b.txt c.txt  
  
JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课后实例/hw2  
$ echo {a,b,c}.txt  
a.txt b.txt c.txt
```

图 6: 实例 2 运行截图

4.3.实例 3：grep 文本搜索

题目：在 hw3 中创建 log.txt（5 行日志，含大小写混合的 error），用 grep 完成四种搜索：查找含 error 的行、统计行数（-c）、忽略大小写（-i）、显示行号（-n）。

```

JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课后实例/hw
$ cat > log.txt << 'EOF'
> INFO:server started
> ERROR:disk full
> info:request ok
> error:timeout
> WARNING:low memory
> EOF

JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课后实例/hw
$ grep error log.txt
error:timeout

JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课后实例/hw
$ grep -c error log.txt
1

JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课后实例/hw
$ grep -i error log.txt
ERROR:disk full
error:timeout

JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课后实例/hw
$ grep -n error log.txt
4:error:timeout

```

图 7: 实例 3 运行截图

4.4.实例 4：重定向与管道

题目：在 hw4 中创建 run.log：第一行用 > 覆盖写入，后两行用 » 追加写入；用 wc -l 统计行数，再用管道把 ls 输出接入 wc -l 统计目录项数。

```

JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课后实例
$ mkdir hw4

JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课后实例
$ cd hw4

JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课后实例/hw4
$ echo "info:start" > run.log

JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课后实例/hw4
$ echo "info:done" >> run.log

JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课后实例/hw4
$ echo "info:extra" >> run.log

JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课后实例/hw4
$ cat run.log
info:start
info:done
info:extra

JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课后实例/hw4
$ wc -l run.log
3 run.log

JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课后实例/hw4
$ ls | wc -l
1

```

图 8: 实例 4 运行截图

4.5.实例 5：变量与文件名空格

题目：在 hw5 中定义变量 name 值为 My File.txt（含空格），用 echo 输出并用该变量创建文件，正确处理文件名中的空格。

```
JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课后实例/hw4
$ cd ../hw5

JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课后实例/hw5
$ name="My File.txt"

JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课后实例/hw5
$ echo "文件名为 $name"
文件名为 My File.txt

JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课后实例/hw5
$ touch "$name"

JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课后实例/hw5
$ ls
'My File.txt'
```

图 9: 实例 5 运行截图

4.6.实例 6：for 循环批量处理

题目：在 hw6 中创建 a.log、b.log、c.log，用 for 循环遍历所有.log 文件，对每个文件打印分隔线、文件名和行数。

```
JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课后实例/hw5
$ cd ../hw6

JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课后实例/hw6
$ echo a > a.log && echo b > b.log && echo c > c.log

JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课后实例/hw6
$ for f in *.log; do
> echo "=== $f ==="
> wc -l "$f"
> done
=== a.log ===
1 a.log
=== b.log ===
1 b.log
=== c.log ===
1 c.log
```

图 10: 实例 6 运行截图

4.7.实例 7：find 精确查找

题目：在 hw7 中创建 a.log、b.log、.secret（隐藏文件）及子目录 sub/c.log，用 find 递归查找所有.log（含子目录）及所有点开头隐藏文件。

```
JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课后实例/hw7
$ touch a.log b.log .secret

JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课后实例/hw7
$ touch sub/c.log
touch: cannot touch 'sub/c.log': No such file or directory

JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课后实例/hw7
$ mkdir sub

JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课后实例/hw7
$ touch sub/c.log

JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课后实例/hw7
$ fine . -name "*.log"
bash: fine: command not found

JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课后实例/hw7
$ find . -name "*.log"
./a.log
./b.log
./sub/c.log

JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课后实例/hw7
$ find . -name ".*"
.
./secret
```

图 11: 实例 7 运行截图

4.8.实例 8：分次提交（模拟真实开发习惯）

题目：在 hw8 中初始化 Git 仓库，分三次提交：创建 main.py 并提交、创建 utils.py 并扩展 main.py 后提交、添加 README.md 后提交，形成 3 条独立 commit（练习禁止单次全量提交）。

```

JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课后实例/hw8
$ git init -b main
Initialized empty Git repository in D:/@作业/系统开发基础/课后实例/hw8/.git/

JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课后实例/hw8 (main)
$ echo 'print("hi")' > main.py

JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课后实例/hw8 (main)
$ git add main.py
warning: in the working copy of 'main.py', LF will be replaced by CRLF the next time Git touches it

JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课后实例/hw8 (main)
$ git commit -m "add:main.py 主程序"
[main (root-commit) 6d88176] add:main.py 主程序
1 file changed, 1 insertion(+)
create mode 100644 main.py

JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课后实例/hw8 (main)
$ echo 'def util(): pass' > utils.py

JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课后实例/hw8 (main)
$ echo 'printf("hi again")' >> main.py

JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课后实例/hw8 (main)
$ git add main.py utils.py
warning: in the working copy of 'main.py', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'utils.py', LF will be replaced by CRLF the next time Git touches it

JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课后实例/hw8 (main)
$ git commit -m "add:utils.py 并扩展 main.py"
[main d35d9c6] add:utils.py 并扩展 main.py
2 files changed, 2 insertions(+)
create mode 100644 utils.py

JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课后实例/hw8 (main)
$ echo "# hw8 练习" > README.md

JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课后实例/hw8 (main)
$ git add README.md
warning: in the working copy of 'README.md', LF will be replaced by CRLF the next time Git touches it

JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课后实例/hw8 (main)
$ git commit -m "docs: 添加 README"
[main 67c6de5] docs: 添加 README
1 file changed, 1 insertion(+)
create mode 100644 README.md

JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课后实例/hw8 (main)
$ git log --oneline
67c6de5 (HEAD -> main) docs: 添加 README
d35d9c6 add:utils.py 并扩展 main.py
6d88176 add:main.py 主程序

```

图 12: 实例 8 运行截图

4.9.实例 9: git stash 暂存未完成的改动

题目：在 hw9 中初始化 Git 仓库并提交 work.txt，修改后不提交就切到新分支 temp 干活，用 git stash 暂存、切回 main 后用 git stash pop 恢复，验证改动完整、stash 列表为空。

```

JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课后实例/hw9
$ git init -b main
Initialized empty Git repository in D:/@作业/系统开发基础/课后实例/hw9/.git/

JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课后实例/hw9 (main)
$ echo "v1" > work.txt

JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课后实例/hw9 (main)
$ git add work.txt && git commit -m "init: work.txt"
warning: in the working copy of 'work.txt', LF will be replaced by CRLF the next time Git touches it
[main (root-commit) 06d4174] init: work.txt
1 file changed, 1 insertion(+)
create mode 100644 work.txt

JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课后实例/hw9 (main)
$ echo "没改完" >> "work.txt"
> ^C

JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课后实例/hw9 (main)
$ echo "没改完" >> "work.txt"

JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课后实例/hw9 (main)
$ git stash
warning: in the working copy of 'work.txt', LF will be replaced by CRLF the next time Git touches it
Saved working directory and index state WIP on main: 06d4174 init: work.txt

JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课后实例/hw9 (main)
$ git switch -c temp
Switched to a new branch 'temp'

JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课后实例/hw9 (temp)
$ echo "temp work" > temp.txt

JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课后实例/hw9 (temp)
$ git add temp.txt && git commit -m "temp: 临时分支工作"
warning: in the working copy of 'temp.txt', LF will be replaced by CRLF the next time Git touches it
[temp a75a89b] temp: 临时分支工作
1 file changed, 1 insertion(+)
create mode 100644 temp.txt

JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课后实例/hw9 (temp)
$ git switch main
Switched to branch 'main'

JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课后实例/hw9 (main)
$ git stash pop
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   work.txt

no changes added to commit (use "git add" and/or "git commit -a")
Dropped refs/stash@{0} (f785e63977000429d237d3540edc27ce84a6cde1)

JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课后实例/hw9 (main)
$ git stash list

JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课后实例/hw9 (main)
$ cat work.txt
v1
没改完

```

图 13: 实例 9 运行截图

4.10.实例 10：撤销修改

题目：在 hw10 中初始化 Git 仓库并提交 file.txt（内容 v1），改坏后未提交，用 git checkout 丢弃未提交改动，恢复文件并确认工作区干净。

```

JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课后实例/hw10
$ git init -b main
Initialized empty Git repository in D:/@作业/系统开发基础/课后实例/hw10/.git/

JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课后实例/hw10 (main)
$ echo "v1" > file.txt

JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课后实例/hw10 (main)
$ git add file.txt && git commit -m "init: file.txt"
warning: in the working copy of 'file.txt', LF will be replaced by CRLF the next time Git touches it
[main (root-commit) ed3dc72] init: file.txt
1 file changed, 1 insertion(+)
create mode 100644 file.txt

JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课后实例/hw10 (main)
$ echo "我改坏了" > file.txt

JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课后实例/hw10 (main)
$ git checkout -- file.txt

JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课后实例/hw10 (main)
$ git status
On branch main
nothing to commit, working tree clean

JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课后实例/hw10 (main)
$ cat file.txt
v1

```

图 14: 实例 10 运行截图

4.11.实例 11：列表 + 代码块 + 公式排版

题目：在 hw11 中编写中文 LaTeX 文档 hw11.tex：无序列表（itemize）与有序列表（enumerate）、一段 C++ 代码块（listings 高亮）、一个带编号公式（equation，二次方程求根公式），用 XeLaTeX 编译两遍。

1 列表练习

1

1 列表练习

无序列表：

- 第一项
- 第二项

有序列表：

1. 第一步
2. 第二步

2 代码块

```
#include <iostream>
int main() { return 0; }
```

3 公式

求根公式 (1):
$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$
(1)

图 15: 实例 11 运行截图

4.12.实例 12：图文排版 + 目录 + 超链接

题目：在 hw12 中编写中文 LaTeX 文档 hw12.tex：自动生成目录（含至少两个 section 各带 subsection）、插入一张本地图片、一个可点击超链接（hyperref），用 XeLaTeX 编译两遍。

目录

1

目录

1 第一节

1.1 第一节

2 第二节

2.1 第二小节

1 第一节

1.1 第一节

这里是正文内容。

2 第二节

2.1 第二小节

图片示例：

Sample Pic

图 1: 示例图片

图 16: 实例 12 运行截图

5. 代码仓库与提交记录

本课程的课上练习（q01-q04）与课后练习代码均已上传至个人 GitHub 仓库，仓库地址：

- GitHub 仓库：<https://github.com/kokona-iiiiu1/sdt-homework>

仓库中的代码采用分次提交。commit 记录的截图如下。

```
JYM@DESKTOP-OPGLD3I MINGW64 /d/@作业/系统开发基础/课堂 (main)
$ git log --oneline --decorate
3766bdc (HEAD -> main, origin/main) add: hw12 图文目录链接
a8e1768 add: hw11 列表代码公式
357d752 add: hw10 撤销修改
841ae04 add: hw9 git stash 暂存
fa8630b add: hw8 分次提交习惯
37fb3aa add: hw7 find 精确查找
8f41038 add: hw6 for 循环批量处理
f231d36 add: hw5 变量与文件名空格
253c335 add: hw4 重定向与管道
81688c4 add: hw1 补充 .log 数据文件
94ed643 fix: .gitignore 不忽略 homework 的 .log
0c9fa5c add: hw3 grep 文本搜索
7a44d49 add: hw2 通配符展开验真
6484cc7 add: hw1 通配符批量操作
1e1790a refactor: 课上练习收进classwork/
86aae9c add: q04 修复并构建一页技术说明
67c4820 add: q03 制造并解决一次合并冲突
9549ab2 add: q02 随机访问日志统计
a061cbd add: q01 含空格文件名的批量整理
```

图 17: c o m m i t 截图

6. 实验总结

本次实验是《系统开发工具基础》第 1 周的内容，围绕 Shell 命令行、Git 版本控制与 LaTeX 排版三个工具链展开。通过课上 4 道检查题和课后 12 个练习实例，我收获良多。

在 Shell 方面，我掌握了文件名中的空格与隐藏文件处理、通配符与花括号展开、重定向与管道、chmod 权限数字法、grep 文本检索、for 循环批量处理、find 递归查找等核心技能。最重要的是建立了管道思想，把一个大任务拆成若干小步骤，用 | 串联，让数据一环一环加工，这也是 Shell 最强大的组合能力。

在 Git 方面，我从零经历了 init、分支、提交、合并的全过程，亲手制造并解决了一次合并冲突，理解了冲突的根源（两个分支改同一文件的同一行）与解决流程（人工裁决 + add + commit）。同时养成了分次提交的良好习惯，并学会用 git stash 暂存未完成工作、用 git checkout 撤销修改。

在 LaTeX 方面，我学会用源码 + 编译的流程产出 PDF，掌握了公式、表格、交叉引用三大核心功能，并理解了交叉引用“编译两遍才不出??”的原理。为支持中文，还学会了使用 XeLaTeX + ctex 组合。

遇到的问题与解决：一是作业系统与本机的网络代理配置导致 git push 时 SSL 握手失败，通过移除代理、改用直连解决了；二是 LaTeX 中文编译需用 XeLaTeX、图片 [H] 定位需加载 float 宏包，编译报错时通过查看日志 (.log) 逐一定位；三是 Git 提交前要检查 git 身份配置与 gitignore（忽略编译产物），避免上传多余中间文件。

总体而言，这次实验让我从“能运行”走向“可复现、可协作、可持续”的工程习惯，为后续每周的工具链学习打下了坚实基础。